Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

11. Develop a POS tagger sensitive to context and training corpus size using HMM and trigram features

Aim: To compare unigram, bigram, and trigram POS taggers and understand how context and training corpus size affect tagging.

```
import nltk
from nltk.corpus import brown
from nltk.tag import UnigramTagger, BigramTagger, TrigramTagger

# Download the Brown Corpus.
nltk.download('brown')

# Load tagged sentences from the Brown news category.
tagged_sentences = brown.tagged_sents(categories='news')

# Choose a training corpus size.
training_size = 500

# Divide the corpus into training and testing sets.
train_data = tagged_sentences[:training_size]
test_data = tagged_sentences[training_size:training_size + 100]

# Train a unigram tagger using individual-word frequencies.
unigram_tagger = UnigramTagger(train_data)

# Train a bigram tagger using the current word and previous tag context.
bigram_tagger = BigramTagger(train_data, backoff=unigram_tagger)

# Train a trigram tagger using two preceding tags as context.
trigram_tagger = TrigramTagger(train_data, backoff=bigram_tagger)

# Evaluate each model on the same test set.
print("Unigram accuracy:", unigram_tagger.accuracy(test_data))
print("Bigram accuracy:", bigram_tagger.accuracy(test_data))
print("Trigram accuracy:", trigram_tagger.accuracy(test_data))

# Test the trigram tagger on a new sentence.
sentence = "the company announced new products".split()

print("\nTagged sentence:")
print(trigram_tagger.tag(sentence))
```

Sample output

```
Unigram accuracy: [calculated accuracy]
Bigram accuracy: [calculated accuracy]
Trigram accuracy: [calculated accuracy]

Tagged sentence:
[('the', 'AT'), ('company', 'NN'), ...]
```

The actual accuracy and tags are produced when the program runs.

Explanation:

• **Unigram tagger:** Assigns tags based on individual words.

• **Bigram tagger:** Uses the preceding POS tag as context.

• **Trigram tagger:** Uses the two preceding POS tags as context.

• **Backoff:** If a higher-order model cannot predict a tag, it falls back to the lower-order model.

To study training corpus size, change `training_size` to `100`, `500`, or `1000` and compare the resulting accuracy. Keep the test set fixed for a fair comparison.

Note that this program implements contextual n-gram taggers, not an HMM. The HMM implementation is given in Programs 9 and 10.